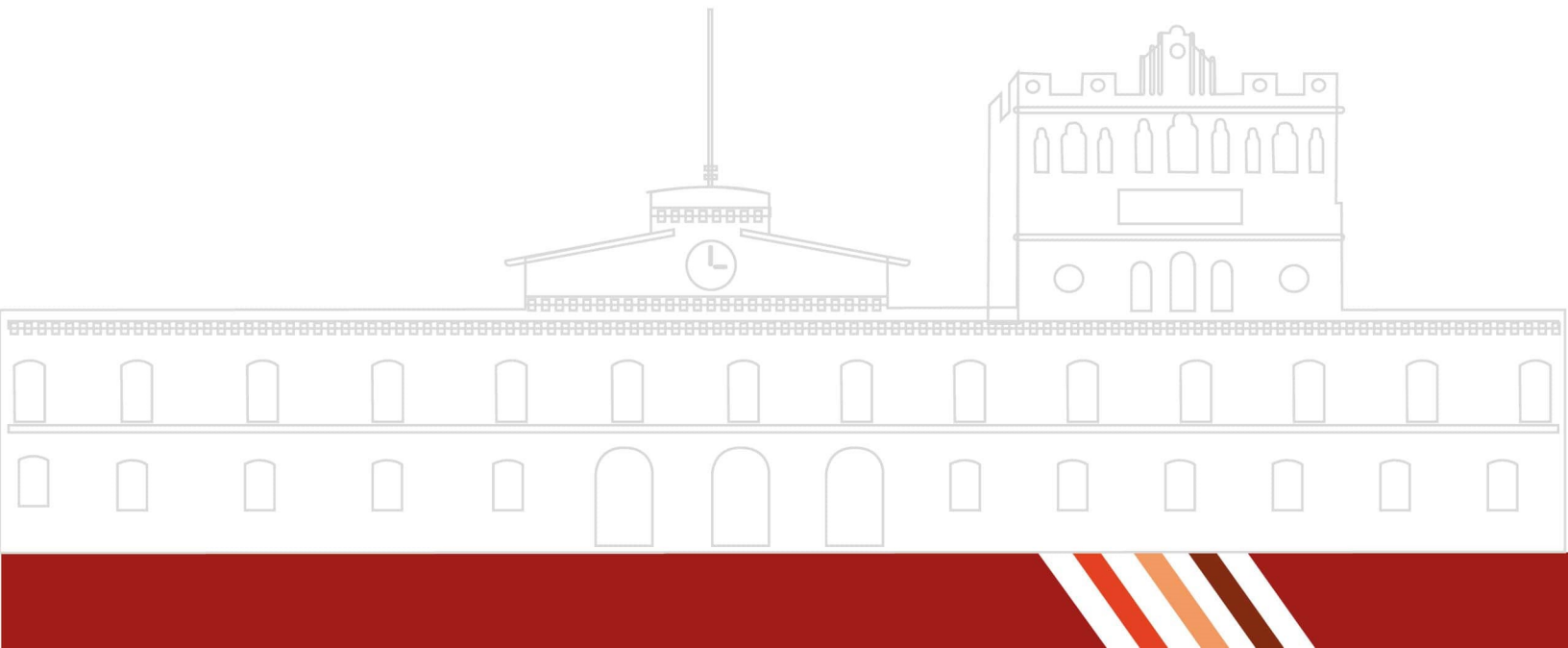


REPORTE DE PRÁCTICA NO. 7

Práctica 7 Análisis sintáctico de elementos de un lenguaje de programación

ALUMNOS:

Gutiérrez Reyes Jasson Axel
Pérez García Alejandro Máximo
Sánchez Barrón Javier Alejandro
Zúñiga Montoya José Antonio



Contents

1	Introducción	3
2	Marco Teórico	3
3	Objetivos	3
3.1	Objetivo General	3
3.2	Objetivos Específicos	3
4	Desarrollo	3
4.1	Librerías, Cabeceras y Variables Externas	3
4.2	Compatibilidad de Formato (El Retorno en Windows)	4
4.3	Analizador Léxico (lexico.l)	4
4.4	Analizador Sintáctico (sintactico.y)	4
5	Resultados	5
6	Conclusiones	5
7	Cuestionario	6
8	Bibliografía	6

Introducción

El análisis sintáctico en lenguajes basados en gramáticas formales resulta esencial para facilitar la comprensión del compilador y abordar de manera más efectiva la validación de estructuras lógicas complejas. Mientras que en prácticas anteriores se analizaron operaciones aritméticas, esta práctica se enfoca en el reconocimiento y validación de estructuras de control de flujo. Específicamente, se construyó un analizador adaptado a la sintaxis del lenguaje Python para la estructura condicional `if`, verificando el correcto uso de los dos puntos (`:`) y la indentación/saltos de línea lógicos.

Marco Teórico

El analizador sintáctico opera sobre una Gramática Libre de Contexto (GLC), que establece las reglas para la formación de las estructuras del lenguaje. En el caso específico de Python, las estructuras condicionales difieren de lenguajes como C o Java al prescindir de palabras reservadas como `then` o llaves `{}`, apoyándose en su lugar en terminaciones con dos puntos (`:`) y bloques definidos visualmente.

El uso de derivaciones (sustitución de símbolos no terminales por cadenas del lado derecho de una producción) permite a herramientas como Yacc construir un autómata de pila capaz de verificar si una cadena de tokens cumple estrictamente con el diseño del lenguaje original.

Objetivos

Objetivo General

Implementar un analizador léxico y un analizador sintáctico utilizando Lex y Yacc para reconocer y validar la estructura condicional `if` (adaptada a Python), capaz de realizar la detección de errores y la validación de gramáticas en un lenguaje de programación de alto nivel.

Objetivos Específicos

- Diseñar un programa en Lex que identifique los tokens asociados a la palabra clave `if`, expresiones condicionales, operadores relacionales y bloques de código.
- Implementar un programa en Yacc que verifique la correcta construcción de la estructura condicional mediante una gramática formal orientada a la sintaxis de Python.

Desarrollo

El proyecto consistió en la creación de dos módulos complementarios. Para asegurar una lectura robusta en entornos Windows, se configuró el analizador léxico para ignorar retornos de carro (`\r`).

Librerías, Cabeceras y Variables Externas

Para la comunicación entre Lex, Yacc y el sistema operativo, se utilizaron las siguientes directivas en C:

- `<stdio.h>` y `<stdlib.h>`: Librerías estándar para el manejo de flujos de entrada/salida (como `printf` y `fopen`) y la gestión dinámica de memoria requerida internamente por los búferes de Lex/Yacc.
- `"y.tab.h"`: Cabecera dinámica generada por Yacc que contiene las definiciones numéricas de los tokens. Es el puente que permite a Lex saber qué valor retornar.
- `extern FILE *yyin`; Puntero de archivo global de Lex. Declararlo en Yacc permite redirigir la lectura hacia nuestros archivos `.txt` de prueba.

- `extern int yylex();` Declaración de la función principal del escáner, invocada repetidamente por el parser.
- `void yyerror(const char *s);` Función de manejo de errores sintácticos disparada automáticamente por Yacc al detectar una violación de la gramática.

Compatibilidad de Formato (El Retorno en Windows)

Durante la fase de pruebas del analizador léxico, se identificó una discrepancia técnica relacionada con la codificación de archivos dependiente del sistema operativo. Mientras que en sistemas basados en Unix (como Linux o macOS) la finalización de una línea se representa con un único carácter `\n` (Line Feed), el entorno Windows inserta dos caracteres invisibles al presionar la tecla Enter: un retorno de carro `\r` (Carriage Return) seguido de un `\n`.

Si el escáner no está preparado para esta dualidad, el carácter `\r` es procesado como un símbolo desconocido, lo que provoca que Lex emita "Errores Léxicos" silenciosos que desfasaban el análisis del bloque de código. Para solucionar esto y hacer el compilador multiplataforma, se añadió una regla de expresión regular explícita en el archivo fuente de Lex: `\r { /* Ignorar */ }`. De esta forma, el autómata desecha el retorno de carro y entrega únicamente el salto de línea limpio a Yacc.

Analizador Léxico (lexico.l)

```
%{
#include "y.tab.h"
#include <stdlib.h>
}%
%option noyywrap
%%
" if"           { return IF; }
[0-9]+(\.[0-9]+)? { return NUMERO; }
[a-zA-Z_][a-zA-Z0-9_]* { return IDENTIFICADOR; }
"==" | "!=" | "<=" | ">=" | "<" | ">" { return OPERADOR_RELACIONAL; }
"="            { return ASIGNACION; }
":"            { return DOS_PUNTOS; }
"("            { return PARENTESIS_IZQ; }
")"            { return PARENTESIS_DER; }
\n             { return SALTO; }
\r             { /* Ignoramos retorno de Windows */ }
[ \t]+         { /* Ignoramos espacios y sangrias */ }
.              { return ERROR_LEXICO; }
%%
```

Analizador Sintáctico (sintactico.y)

```
%{
#include <stdio.h>
#include <stdlib.h>
extern FILE *yyin;
extern int yylex();
void yyerror(const char *s);
int linea = 1;
}%
%token IF NUMERO IDENTIFICADOR OPERADOR_RELACIONAL ASIGNACION
%token DOS_PUNTOS PARENTESIS_IZQ PARENTESIS_DER SALTO ERROR_LEXICO
```

```

%%
programa: /* Vacio */ | programa linea_codigo | programa estructura_if ;
linea_codigo: SALTO { linea++; } | instruccion SALTO { linea++; } ;

estructura_if:
    IF condicion DOS_PUNTOS SALTO bloque_instrucciones {
        printf("->-EXITOSO-(Linea-%d):-Estructura-'if'-de-Python-valida.\n", linea);
    }
    | error SALTO {
        yyerrok;
        printf("->-ERROR-SINTACTICO-(Linea-%d):-Condicion-o-instruccion-mal-formada.\n", linea);
        linea++;
    }
    ;

condicion:
    valor OPERADOR_RELACIONAL valor
    | PARENTESIS_IZQ valor OPERADOR_RELACIONAL valor PARENTESIS_DER
    ;

valor: IDENTIFICADOR | NUMERO ;

bloque_instrucciones:
    instruccion SALTO { linea++; }
    | bloque_instrucciones instruccion SALTO { linea++; }
    | instruccion { /* Acepta instruccion al final del archivo EOF */ }
    | bloque_instrucciones instruccion { }
    ;

instruccion: IDENTIFICADOR ASIGNACION valor ;
%%
void yyerror(const char *s) {}
int main(int argc, char **argv) {
    if (argc > 1) { yyin = fopen(argv[1], "r"); }
    yyparse();
    return 0;
}

```

Resultados

Al ejecutar el programa compilado, se validaron dos archivos: uno con sintaxis de Python correcta y otro introduciendo errores comunes provenientes de otros lenguajes.

Conclusiones

La práctica demostró la flexibilidad de las herramientas de generación de compiladores para adaptarse a las reglas específicas de distintos lenguajes. Se comprobó que el analizador sintáctico rechaza eficientemente estructuras que, aunque léxicamente contienen palabras válidas, violan el orden gramatical estricto definido en la notación BNF, como la ausencia del token `DOS_PUNTOS` tras evaluar una condición.

```

C:\Flex Windows\EditPlusPortable\Lex\NewAnl>mi_parser.exe correcto.txt
--- INICIANDO ANALISIS SINTACTICO PYTHON (IF) ---
-> EXITOSO (Linea 3): Estructura 'if' de Python valida.
-> EXITOSO (Linea 5): Estructura 'if' de Python valida.
--- ANALISIS FINALIZADO ---

C:\Flex Windows\EditPlusPortable\Lex\NewAnl>mi_parser.exe incorrecto.txt
--- INICIANDO ANALISIS SINTACTICO PYTHON (IF) ---
-> ERROR SINTACTICO (Linea 1): Condicion 'if' o instruccion mal formada.
-> ERROR SINTACTICO (Linea 4): Condicion 'if' o instruccion mal formada.
--- ANALISIS FINALIZADO ---

C:\Flex Windows\EditPlusPortable\Lex\NewAnl>

```

Figure 1: Ejecución del programa.

Cuestionario

1. **¿Cuáles son las principales ventajas de usar gramáticas de contexto libre para definir estructuras como `if...then` en lenguajes de programación?**
Permite establecer jerarquías precisas de manera declarativa y modular, evitando el uso de algoritmos estáticos complejos. Además, facilita la detección y resolución de ambigüedades lógicas por parte del autómata de pila generado.
2. **¿Qué tipo de errores fueron detectados en los ejemplos incorrectos y cómo influyeron las gramáticas definidas para lograr su identificación?**
Se detectaron violaciones a la estructura esperada por Python: la omisión de los dos puntos (`:`) al final de la condición y la inclusión del token `then`. La gramática influyó directamente ya que la producción `estructura_if` exige secuencialmente el token `DOS_PUNTOS`; al no encontrarlo, el autómata rechaza la instrucción y ejecuta la regla de recuperación de error.
3. **¿Cómo contribuye el uso de derivaciones por la izquierda o por la derecha en la construcción de un árbol sintáctico para validar estructuras condicionales?**
Las derivaciones determinan el orden en el que se sustituyen y evalúan los bloques lógicos. En estructuras condicionales anidadas, establecer una estrategia clara de derivación ayuda a resolver problemas clásicos como el "dangling else", asegurando que el compilador asocie inequívocamente cada instrucción a su respectivo bloque de control.
4. **¿Qué ajustes serían necesarios en la gramática para permitir el reconocimiento de estructuras condicionales más complejas, como `if...then...else`?**
En el caso de Python, sería necesario que Lex reconozca nuevas palabras reservadas como `else` y `elif`. En Yacc, se debería expandir la regla principal para incluir ramas opcionales, por ejemplo: `IF condicion DOS_PUNTOS SALTO bloque ELSE DOS_PUNTOS SALTO bloque`, asegurando que el análisis sintáctico cubra todas las bifurcaciones lógicas posibles.

Bibliografía

- Aho, A. V., Ravi, S. A. V., & Ullman, J. D. (1998). *Compiladores: Principios, técnicas y herramientas*. Addison Wesley Longman. México.
- Giró, J., Vázquez, J., Meloni, B., Constable, L. (2015). *Lenguajes formales y teoría de autómatas*. Editorial Alfaomega. Argentina.